RadixHuskySort: A Linear Second Phase for Proxy-Key Sorting

R C Hillyard* Yunlu Liaozheng* Sai Vineeth K R*

Abstract. A common computational model for sorting algorithms is based on the numbers of comparisons and/or exchanges they perform. We argue that a better yardstick is the total number of array accesses, which, for a divide-and-conquer sort, falls into a linearithmic phase and possibly a linear one: moving work out of the first and into the second reduces the total. This is particularly true where comparison itself is expensive likely leading to a reduction in running time as well. The mechanism we present is based a 64-bit code, approximately order-preserving, that stands as a proxy for each element; we call it a husky code. Its effect is to extract each element's sort key once, or at most twice, rather than at each level of recursion. Earlier work sorted such codes with quicksort. We replace that with a linear-time radix sort, making the second phase linear too, and describe a parallel variant of it. Measured against Java's system sort for objects (*Arrays.sort*) — integers, arbitrary-precision numbers, dates, tuples, English and Chinese text, Chinese personal names in pinyin order, and, by way of example, a large number of municipal records ordered by a composite key — RadixHuskySort is faster in every case, by 3.6x to 6.8x at the largest size measured for each. The margin is widest where the ordering is expensive to evaluate and the encoding is perfect; and, where the encoding is imperfect, we measure directly what the resulting cleanup pass costs. Two things overtake the serial sort: on English words a tuned MSD radix sort, and, where cores are free, the JDK's own parallel sort — though parallel against parallel our own variant leads that baseline wherever a comparison is expensive.

1 Introduction. With the vast increase in the size of the data world, efficient computing algorithms play a crucial role in processing data. From the earliest days of computing, sorting has been one of the most studied areas of research even though it appears to be a straightforward and familiar topic. And for good reason. Binary search, which depends on sorting, is perhaps the most significant optimization technique that systems can use. The rise of data volume and complexity has increased the need for new approaches to traditional sorting techniques. Within the realm of comparison-sort algorithms, many aspects of the algorithm can be significant: stability, adaptability, extra memory, partitioning, merging, exchanging (swapping), copying, and, of course, the comparison itself.

Counting memory operations rather than only comparisons is an established idea, and not one we claim as new. Three lines of work bound ours: the external-memory (I/O) model of Aggarwal and Vitter [1], which counts block transfers between disk and memory; the cache-oblivious algorithms of Frigo et al. [8], which reach near-optimal memory-hierarchy behaviour without advance knowledge of the cache parameters; and, closest to our own concern, LaMarca and Ladner's study of how caches shape the relative performance of sorting algorithms [13]. All three are chiefly concerned with data too large to fit in memory, or in cache, at all. Our array-access count is a simpler, single-level analogue: we count raw array reads and writes as a proxy for real cost, motivated by the observation that comparing two heavyweight objects — two *String*s, say — touches more memory, and more likely uncached memory, than comparing two 64-bit primitives.

Radix sort's independent per-digit counting-sort passes are, taken alone, well suited to parallelization across CPU threads, a property already established for radix sort generally, and § A.8 describes and benchmarks a parallel variant of RadixHuskySort (§ 3.2). Those passes, however, are not where a proxy-key sort spends its time: the encoding and the cleanup dominate on the corpora we measure. We have since parallelized the encoding, and provide the cleanup in an opt-in parallel form, so that three of the four phases can run in parallel; only the permutation remains sequential. Measured parallel against parallel, the result is then size- and corpus-dependent rather than a uniform win or loss, and we report it that way (§ A.7). § 7 gives the figures, and is careful to distinguish what bounds our implementation from any claim about the mechanism, which we do not make. We do not make even the narrow claim for QuickHuskySort's own comparison-based steps: Introsort's partitioning and Timsort's adaptive run-detection are both substantially

*College of Engineering, Northeastern University, 360 Huntington Avenue, Boston, Massachusetts 02115, USA (r.hillyard@northeastern.edu, liaozheng.y@northeastern.edu, kandappareddigari.s@northeastern.edu).

more data-dependent (less independent) and sequential in character than radix sort's fixed per-digit passes, so parallelizing them well is a separate, harder problem that this paper does not address.

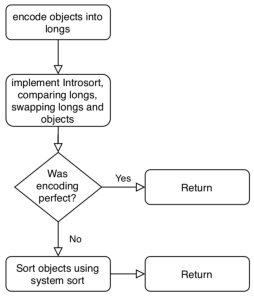
While sorting appears to be a settled topic, new improvements are occasionally made: [21, 17, 12, 15, 4]. Indeed, there is an abundance of sorting techniques suitable for a myriad of different use cases. Nevertheless, there is still room for improved versions in regard to computation, memory, time, and space complexity. In a recent paper [22], the authors conclude that quicksort is the fastest general-purpose sorting algorithm, especially when appropriate care is taken to avoid the quadratic worst-case with several other improvements (e.g., dual-pivot quicksort) [21] over the traditional version [11]. However, quicksort does not perform as well as some variations of merge sort, especially Timsort [17], when the dataset is partially sorted. Timsort, a hybrid stable sorting algorithm derived from merge sort, is the default object-sorting utility in the Python and Java libraries.

The husky code was introduced by Hillyard et al. [10], together with QuickHuskySort, which sorts an array of objects by sorting an array of their codes and then repairs any remaining inversions among the objects. That work established the basic result — encoding pays for itself for types whose ordering is expensive to evaluate, among them *String*, *LocalDateTime*, *BigInteger*, *BigDecimal* and many user-defined types of the same kind — and it is the starting point of this paper rather than its subject. Two things it left undone define the work here. Its step 2 was a comparison sort, so the encoding made each comparison cheap without reducing how many there were. And the cost of the cleanup pass, though analysed in terms of the proportion of inversions that remain after step 2, was never measured: it reports no calculated value for p_{crit}. The contributions here answer both. RadixHuskySort, or RHSort (§ 3.2), replaces step 2 with a linear-time radix sort on the same codes; we also parallelize its digit passes, which turns out not to pay and is reported as such (§ A.8); § 3.4 measures what the cleanup pass costs. Alongside those, an empirical comparison against the classic string sorts rather than a theoretical one (§ 3.2), and a case study on real rather than generated data (§ 6.3). All results reported here were measured anew.

We review existing methods of sorting in Section 2 and go through the HuskySort algorithm in Section 3. Implementation options and their likely impact are explored in Section 4. We describe our Test Cases in Section 5 and present results in Section 6, with outcomes summarized in Section 7.

2 Background. Two comparison sorts bracket the design space, and HuskySort's three-step strategy

Figure 3.1: HuskySort Flow



follows from the contrast between them. Quicksort is fastest in practice on unordered data, its in-place partitioning and sequential access outweighing merge sort's guaranteed bound, but it is unstable and its worst case is quadratic unless mitigated — as Introsort does, by falling back to heapsort past a recursion-depth threshold [16]. Timsort takes the opposite trade: guaranteed $\mathbf{O}(N \log N)$ and stable, at the cost of extra space and of being no faster than $\mathbf{O}(N \log N)$ on unordered input, but adaptive, such that where the number of inversions I is small, its cost falls towards $\mathbf{O}(N)$. Husky coding exploits both ends of that contrast. Step 2 hands its sort cheap-to-compare, effectively unordered longs, which is quicksort's best case and, as § 3.2 shows, a radix sort's natural input; step 3 hands Timsort an array already close to sorted, the one regime in which its adaptivity pays. Standard complexity and stability characteristics for the algorithms named here are tabulated in § A.2.

3 Algorithm.

3.1 Overview. The strategy of HuskySort (see Flow 3.1) is as follows, assuming the dataset is presented as an array xs of type X:

Step 1:
- a Create a $long[]$ array ($longs$) of the same size as xs;
- b for each element x of xs, set the corresponding element of $longs$ to $x.huskyCode()$.

Step 2: Sort the $longs$ array — with dual-pivot quicksort in the original algorithm, with an insertion-sort cutoff and the heapsort fallback of § 2 — in such a way that when elements i, j of $longs$ are exchanged, then elements i, j of xs are also exchanged;

Step 3: Finally, when step 2 is finished and, *if necessary*, we run Timsort on the array xs to ensure that its elements are truly in order.

Step 3 is needed only where the encoding is imperfect. A *perfect* encoding leaves no inversions at all, so

that sorting the codes has already sorted the elements and step 3 can be skipped; § 3.3 states the condition precisely and § 3.4 measures what an imperfect one costs.

For a worked example of a husky coding, see Algorithm 3

3.2 RadixHuskySort. Step 2's quicksort is $\mathbf{O}(N \log N)$, but the codes it sorts are fixed-width 64-bit values, and radix sort can sort k-bit keys in $\mathbf{O}(N \cdot \frac{k}{b})$ time using b-bit digits — linear in N for any fixed digit width b. So step 2 can be linear too. This section describes RadixHuskySort, which makes it so (Algorithm 1).

Radix sort's linear-time advantage rests on a further assumption beyond fixed key width: that a digit can be turned into an array index in $\mathbf{O}(1)$ time, so that counting and scattering never need to compare two digits directly. A b-bit digit of a husky code satisfies this trivially — it is extracted by a single shift and mask, and used directly as an index into a 2^b-bucket counting array. The original *Comparable* type generally does not: without first reducing it to such a code, radix sort applied directly to a *String*, a *Tuple*, or a case class would need some other way to decide which of 2^b buckets a given key belongs to, and whenever the natural notion of a "digit" for such a type is not already a small, dense integer, that decision falls back to *equals* (and, for a hash-based bucket structure, *hashCode*) on the type itself — an $\mathbf{O}$(size of the value) operation, not an $\mathbf{O}(1)$ one. Husky encoding is what removes this cost from every one of RadixHuskySort's $\frac{64}{b}$ digit passes, not merely from the final comparison: once the one-time $\mathbf{O}(N)$ encoding step has run, *equals* and *hashCode* on the original type are not called again until the optional cleanup pass.

A naive adaptation — swapping the object reference alongside each long at every exchange within every digit pass, exactly as step 2's Introsort swaps at every partition exchange — would incur $\mathbf{O}(N)$ reference movements per digit pass, or $\mathbf{O}(N \cdot \frac{64}{b})$ movements in total. Since $\frac{64}{b}$ is typically much smaller than $\log N$ at the sizes we tested, this would still usually beat Introsort's swap cost, but it needlessly reintroduces exactly the kind of payload-movement overhead this whole design exists to avoid, and least-significant-digit (LSD) radix sort's per-pass counting-sort structure makes a better option available.

Instead, RadixHuskySort's step 2 carries only an *int[]* index array through the LSD digit passes (Algorithm 1), alongside the (unmodified) long array; each pass is a textbook stable counting sort [6] keyed on one b-bit digit of the (already-computed) husky code: a histogram of how many indices carry each of the 2^b digit values, a prefix sum turning that into a starting offset per value, then a scatter of the indices into their new order — reordering the index array rather than the codes or the objects themselves. Once all $\frac{64}{b}$ passes are complete, the index array holds the sorting permutation, and a single $\mathbf{O}(N)$ pass builds the final object array by reading *xs[index[i]]* for each i. This confines all object-reference movements to one pass in total, rather than one pass per digit.

Because LSD radix sort's per-digit counting sort treats keys as unsigned, and husky codes are signed 64-bit values whenever the underlying domain can be negative (e.g., *Integer*, *Long*, or dates before a chosen epoch), each code is first transformed by $code \oplus Long.MIN_VALUE$ — flipping only the sign bit — before the digit passes, which places negative codes before non-negative ones under unsigned digit comparison without disturbing relative order within either group; the same transform is idempotent to reverse, so no separate un-flip step is needed once the permutation is built from the transformed codes. Whether a given *HuskyCoder* can actually produce negative codes is checked rather than assumed, since several codings (e.g., Unicode strings) never do.

The digit width b is a tunable parameter, trading off the number of passes ($\frac{64}{b}$) against the size of the per-pass counting array (2^b buckets, which must fit comfortably in cache for each pass to stay fast); § 6.3 reports how this trade-off plays out empirically across data types and array sizes. Step 3 (the cleanup pass, run only if the coding is not provably *perfect*) is unchanged regardless of which algorithm sorts the codes in step 2 — radix sort is a drop-in replacement for that one step, not a change to the overall three-step strategy.

This parameterization also addresses a related limitation: assuming 64-bit words throughout may not be appropriate for comparison on different key sizes. Quicksort's asymptotic behaviour does not depend on key width at all — an $\mathbf{O}(N \log N)$ comparison sort costs the same whether the keys being compared are 32 bits or 128 bits wide, since only the (fixed-cost) comparison itself touches the key's actual bits. Radix sort, by contrast, makes the key width an explicit, first-class parameter: a k-bit key simply takes $\frac{k}{b}$ passes instead of $\frac{64}{b}$, with no change to the algorithm itself. So while nothing about the discussion above depends on 64-bit words specifically, the RadixHuskySort generalizes to other key widths more directly than a comparison-based sort does — if a future encoding needed, say, 128 bits to capture more of an object's structure, the same radix sort would apply unchanged, with twice as many passes.

Width alone does not buy perfection, though: only a bounded domain admits a perfect fixed-width code at all, and a coder limited by its own information rather

than by its width — pinyin, whose few hundred syllables leave true homonyms at any width — gains nothing from extra bits. Nor are those passes free, and what they buy is all-or-nothing, since step 3 is skipped only for a perfect encoding: the measured shares put a single pass at roughly a seventh of RHSort's running time against a cleanup pass of at most a sixth (§ 3.4), so the trade pays only where one or two further digits would reach perfection and not at all where they would merely approach it.

Radix sort applied to strings specifically has its own substantial, and older, literature, distinct from the general fixed-width-key radix sort described above. Three-way radix quicksort (also called multikey quicksort) and MSD (most-significant-digit) radix sort for strings were both popularized by Bentley and Sedgewick's classic treatment [5], and burstsort [18] improves cache behaviour further by building a small trie to bucket strings before sorting. These string-specialized algorithms sort variable-length strings directly, character by character, without a separate fixed-width encoding step. RadixHuskySort's contribution is different in kind, not degree: it is not a string-sorting algorithm, but a general mechanism applicable to any *Comparable* type with a 64-bit husky encoding (strings among them), at the cost of the encoding's own fixed capture window (§ 3.4 above) and a fallback cleanup pass whenever that encoding is imperfect.

That framing carries a consequence which Table 6.2 bears out, and which we state rather than leave to be noticed: the string rows of that table are its most variable by a wide margin. They span from 3.4x down to 1.6x, nearly the whole of its range below dates, where every other row falls between 2.1x and 3.6x. Three factors account for the spread, and they compose. The advantage grows with the cost of the type's native comparison, since that is what the encoding replaces; it grows with the perfection of the encoding, since an imperfect one is paid for by the cleanup pass; and it shrinks in the presence of algorithms specialised to the type. Dates are the favourable extreme on all three counts — a provably perfect coding, no cleanup pass at all, and no specialised competitor — and yield the largest margin in the table at 5.5x. The string rows show those same three factors varying *within* a single data type, which is the more instructive demonstration. Chinese words are favourable on the first two counts — an expensive Unicode comparison, captured well enough by the encoding — and sit fourth of eleven, above all but three of the non-string rows. Chinese personal names are unfavourable on the second: the comparison is more expensive still, a table lookup per character, but the pinyin coding captures far less of the key than the Unicode coding does, so ties are common and the cleanup pass does real work rather than none (§ A.11 accounts for the difference). The more expensive comparison is paired with the weaker encoding, and this is the weakest result we report. English words are unremarkable on the first two counts and distinctive on the third, being the one case in this paper with a mature specialised literature ranged against it.

Strings are unfavourable on the third count in a way no other type in this paper is. Half a century of specialised string sorting exists, and a general mechanism should not be expected to beat it on its own ground. We compared RadixHuskySort against both of the classic string sorts named above, under JMH, on English and Chinese text, in code-point order so that every sort compared performs the same task. Both baselines are our own implementations, each tuned so that its fallback below the partitioning cutoff allocates nothing and compares from the depth the partitioning has already established. Against three-way radix quicksort RadixHuskySort is faster at every size tested, by 1.3–2.4x on English words and 3.1–4.0x on Chinese words, with non-overlapping 99.9% confidence intervals throughout. Against MSD radix sort the result goes the other way at scale, though not by much: on English text the two are indistinguishable at $N = 32{,}000$, and MSD is faster by 1.20x at $N = 200{,}000$ and by 1.06x at $N = 1{,}000{,}000$. The advantage at $N = 200{,}000$ is the robust one. At $N = 1{,}000{,}000$ three runs on the machine of record give 1.00x, 1.05x and 1.06x, so at the largest size we measure the two algorithms are level to within the spread between runs, and the direction should be read from the middle size rather than the largest. We give this result because it locates the boundary rather than obscuring it. MSD earns it on a corpus that suits it, and the limits that keep it off the others are ours rather than the algorithm's: our alphabet reserves 256 positions for characters beyond ASCII, which English text fits and neither Chinese corpus does, and the collation we gave it is code-point order rather than pinyin. Neither ceiling is inherent — a wider alphabet and a pinyin-aware MSD are both implementable, and we did exactly that for the multikey baseline below — so the absent Chinese rows record work we did not do, not a limitation of MSD radix sort. RadixHuskySort reaches every corpus in this paper through one encoding, with no per-alphabet provision and no per-type implementation. That is the claim being made, and the English result bounds it without contradicting it. A direct empirical comparison against burstsort remains future work; it is a trie-based, cache-conscious algorithm designed to beat both baselines used here on exactly this workload, and we have not implemented it.

Code-point order is not a meaningful comparison for Chinese personal names specifically — it is simply the wrong order for that use case, not merely a different one. Leaving the word corpus in that order is deliberate: a directory of people has one conventional ordering and it is by pinyin, where a word list has several — by radical, stroke count, pinyin or frequency, according to purpose — so no single ordering is the right one to hold it to, and code-point order keeps that row comparable with the English one. A fair comparison for the names requires a pinyin-aware MultikeyQuicksort. We extended it with the same syllable-then-tone character key the pinyin encoding of § 3.2 already uses, restricted the comparison to the three pinyin-aware sorters, and re-ran it. RHSort is faster than the repaired multikey quicksort by 1.54x, 1.80x and 2.08x at $N = 32{,}000$, 200,000 and 1,000,000, and QuickHuskySort by 1.33x, 1.49x and 1.27x. Two independent runs on the machine of record agree to within seven hundredths of those multipliers. Note that the RHSort margin grows with N while QuickHuskySort's does not: the linear second phase keeps paying as the array grows where a linearithmic one cannot, which is the same pattern the natural-order corpora show and the clearest form in which this corpus shows it.

Algorithm 1: RadixHuskySort.sort. The digit passes reorder only *index*, an *int[]*, so the payload is permuted once at the end rather than once per digit.

```
   Input: xs as an array of object X which
          extends Comparable<X>
   Input: b as the number of bits per digit
   Output: xs, sorted
1  coding = huskyCoder.huskyEncode(xs) ;
    // Algorithm 2
   // flip the sign bit, so unsigned digit
      order matches signed order
2  biased[i] = coding.longs[i] XOR
    Long.MIN_VALUE, for each i;
3  index = [0, 1, ..., xs.length - 1];
4  for digit = 0 to (64 / b) - 1 do
5      index = stableCountingSortPass(biased,
         index, digit, b);
   // the only pass that moves object
      references
6  source = copy of xs;
7  xs[i] = source[index[i]], for each i;
8  if not coding.perfect then Arrays.sort(xs);
9  return xs;
```

3.3 Discussion of Husky Encoding. Step 1b requires the definition of a 64-bit husky code (refer to Algorithm 2). Its defining property is monotonicity: if x and y compare equal then $x.huskyCode() = y.huskyCode()$, and if $x > y$ then $x.huskyCode() \geq y.huskyCode()$ — the latter holding with probability $1 - p$ and failing with probability p. Note that the relation that matters is *compareTo*, not *equals*: the two differ for *BigDecimal*, whose coder is *longValue*, and it is the ordering the sort must respect.

The two ways the second condition can fail differ. If $x > y$ but $x.huskyCode() < y.huskyCode()$, the codes are positively wrong and step 2 will invert the pair. If $x > y$ but $x.huskyCode() = y.huskyCode()$, the codes say nothing about the pair at all, so whatever order step 2 leaves it in is arbitrary with respect to the true one, and it may be inverted. Only step 3 can settle either case.

We call such a code *approximately order-preserving*, or p-order-preserving where the fraction matters. The relaxation is what the method trades on: an *exactly* order-preserving fixed-width code exists only for bounded domains, whereas allowing a fraction p of pairs to come out wrong buys a 64-bit code for domains admitting no exact one, at the price of a cleanup pass. Order-preserving compression for sorting keeps exactness and pays for it in variable-length codes [2, 14]; we fix the width and pay in p instead.

For a successful encoding, the value of p, the probability of a potential inversion, should be less than some critical probability, p_{crit}. Leaving a more detailed discussion of p_{crit} until later, we note that, for some domains of X, we can definitely assert that $p = 0$ — which is the *perfect* encoding of § 3.1, stated precisely. In such a case we can skip step 3, because with $p = 0$ no inversions remain after sorting the husky codes. An example of such a domain is date-time values with a resolution of one nano-second over a period of 128 years. For strings of English case-independent letters (no punctuation or numbers), i.e., 5-bit ASCII, words of 12 (or fewer) characters in length can also be coded uniquely. For case-dependent strings (6-bit ASCII), we can encode perfectly if the lengths are all 10 characters or less. In step 3, it might be acceptable to use insertion sort instead of Timsort. In any event, assuming that the huskyCode is as accurate as required, then there will be very few inversions (elements out of place) remaining after step 2. This implies that the second sort can be performed in very close to linear time. The results have been very satisfactory. For example, taking an array of 1,000,000 English words drawn randomly from a corpus of 275,333 [9], QuickHuskySort sorts in 698 ms against the system sort's 1,121 ms, a factor of 1.61 (for the full

data, please refer to table 6.1).

3.4 Discussion of p_{crit}. When $p > 0$, there is a finite probability of any pair of elements remaining inverted after step 2. For an array of N randomly ordered elements, the average number of inversions before sorting is $X = N(N-1)/4$. The number of inversions remaining after step 2 is simply pX. And, for an adaptive insertion sort, the time to re-sort the array will be $t = k(N+pX)$ where k is some system-dependent constant — N to scan, plus one move per inversion that remains. The total time to sort the array is made up of these three time-consuming steps where k_1, k_2, k_3 are system- and algorithm-dependent constants:

Step 1: T_1 is the time to encode the elements:

$$T_1 = k_1 N$$

Step 2: T_2 is the (average) time for step 2, sorting the husky code array while permuting the element array alongside it, with a comparison sort:

$$T_2 = 2k_2 N \ln N$$

Step 3: T_3 is the time for the cleanup pass (only required if $p > 0$), in the adaptive insertion sort's form:

$$T_3 = k_3(N + pX)$$

We in fact use Timsort, whose cost is not $N + pX$ but $\mathbf{O}(N + N \log r)$ in the number of ascending runs r (Table A.1); the two forms meet at $r = 1$, which is the case measured below, and § A.5 settles the choice between them. Provided that the total time $T = T_1 + T_2 + T_3$ is less than the time required to use Timsort on the original element array, the method wins. This same three-step decomposition ($T_1/T_2/T_3$, i.e., encode/sort/cleanup) is revisited in § 5.2 in more concrete terms — counting array accesses directly rather than folding everything into abstract system-dependent constants k_1, k_2, k_3 — and extended there to cover RadixHuskySort (§ 3.2) as a fourth alternative for the T_2 step. The actual values of p_{crit} are very much dependent on the machine running the sort. We have not calculated a typical value for p_{crit} but, even for the worst-case: Chinese characters using UTF8 encoding, we still find very significant gains, reported later in Table 6.1.

Of the three terms, T_3 can be isolated experimentally. Two benchmarks over the same two hundred thousand municipal records (§ 6.3) differ only in which coder they are given. Both compute identical codes; one coder declares itself perfect, so the sort skips step 3, while the other leaves that declaration at its default and the pass runs. The encoding for these records is perfect, so the pass that runs has provably nothing to correct.

Its cost is predictable in advance. Timsort over a fully ascending array detects a single run and stops, performing $N - 1$ comparisons and no moves, so T_3 at $p = 0$ is exactly $N - 1$ invocations of the element's own comparator — here assessor's block, then lot, then filing date, which is precisely the comparison the husky encoding was introduced to avoid. The cleanup pass reintroduces one full composite comparison per element, having just removed it from all $\mathbf{O}(N \log N)$ of them.

Measured on those records the pass costs between a sixteenth and a quarter of the total, rising by a factor of three to four between $N = 32{,}000$ and $N = 100{,}000$ and then levelling off where the records stop fitting in cache. § A.9 gives the per-size figures, the evidence for that reading, and what becomes of the share as N grows.

Two conclusions follow. First, k_3 is not a constant: it varies more than fourfold across three sizes on one machine, so no single p_{crit} can be quoted even for one type on one machine. Second, that cost is fixed rather than proportional to the work it does. An encoding that can be declared perfect skips the pass entirely; one that cannot pays $N - 1$ full composite comparisons — up to a quarter of the running time — before correcting a single genuine inversion.

3.5 Why husky coding works. The rationale is set out in full by Hillyard et al. [10]; we restate only what the rest of this paper depends on. A 64-bit word holds enough of most keys to place elements close to their final order, and comparing two longs costs far less than following two object references and running a type-specific *compareTo* — so the encoding trades one $\mathbf{O}(N)$ pass for cheaper comparisons throughout the $\mathbf{O}(N \log N)$ one. The code need not be perfect, only good: whatever inversions it leaves are the cleanup pass's problem, and § 3.4 measures what they cost.

Perfection has a precise limit for strings. Packing four 16-bit characters fills the word (Table 4.1 gives the constants for both string codings), but the packed result is then shifted right by one bit, because any string whose first character is `0x8000` or above — common among Chinese characters — would otherwise set the sign bit and encode as a negative *long*, corrupting numeric comparison. The shift costs the least significant bit of a true fourth character; for three characters or fewer that bit is padding, which is why the Unicode coding is declared *perfect* up to three characters and not four.

Whether a coding was in fact perfect is known before the cleanup pass runs: a constant-time lookup for non-sequence types, and for strings a scan that stops at the first string too long to encode perfectly. Where it

was perfect, no inversions remain and the pass is skipped altogether.

Algorithm 2: huskyEncode. Perfection depends on the data, not only on the coder, so it is settled as the codes are computed. *perfectFor* tests one element in constant time: a length comparison for a string, the coder's own constant where perfection is data-independent.

Input: *xs as an array of object X*
Output: a *Coding: the codes, and whether they are perfect*

```
1 result = new long[xs.length];
2 perfect = true;
3 for i = 0 to xs.length - 1 do
4     result[i] = huskyEncode(xs[i]);
5     if perfect then perfect = perfectFor(xs[i]);
6 return new Coding(result, perfect);
```

Algorithm 3: A composite husky coder: *PermitCoder*. *encodeString* packs *width* symbols of *bits* each, right-padding with zero.

Input: *permit, with fields block, lot, filedDate;*
Output: a 60-bit code, order-preserving for *permit.compareTo*

```
// fields most-significant first, in
   compareTo order
1 r = encodeString(block, 5, 5,
   BLOCK_ALPHABET);
2 r = r ≪ 24 | encodeString(lot, 4, 6,
   LOT_ALPHABET);
3 return r ≪ 11 | daysSinceEpoch(filedDate);

// codeOf(x, alphabet), used by
   encodeString for each symbol
4 if alphabet contains x then
5     return 1 + index of x;  // 0 is left for
       padding
6 else
7     return count of symbols < x;  // weaken,
       never invert
```

4 Implementation. The implementation is available at https://github.com/rchillyard/HuskySort.

4.1 System Environment. Three machines appear in this paper, and § A.3 specifies all three. Machine A ran the JDK 1.8.0_152 comparison-sort benchmarks of § 5.2, the choice of cleanup sort among them (§ A.5); machine B, on different hardware and two JVM generations later, ran the radix-sort and JMH development work. Machine C is the machine of record: an independently administered cloud instance, confirmed idle before each run, and the source of every figure quoted in this paper. It is ARM-based like machine B but differs from it in core design, vendor, core count and operating system.

Table 4.1: String-encoding constants

Constant	Unicode	ASCII
Bit width per character	16	7
Max length (64 / bit width)	4	9
Mask	`0xFFFF`	`0x7F`

Reporting all three lets a reader check directly whether the qualitative finding generalizes across machines and JVM versions rather than being an artifact of one environment. It does: QuickHuskySort, and now RHSort, beat the system sort on all three, which differ in vendor, instruction set, core design and JVM generation.

5 Test Case and Analysis.

5.1 Data Source. The English and Chinese strings come from the Leipzig Corpora Collection [9], taking the appropriate *sentences.txt* file and splitting each line on whitespace and punctuation. Chinese personal names, used for the pinyin-order comparisons of § 6.3, are a separate corpus [19]. The building-permit records of § 6.3 are 198,900 permits filed between January 2013 and February 2018, taken from San Francisco's published record [7] and ordered by a composite key of assessor's block, lot and filing date; only those three columns are retained, and the data is used under the Open Data Commons Database Contents License. They are included because their distribution is the city's rather than ours.

Every other dataset in this paper is generated: pseudo-random numbers, dates and tuples.

5.2 Analysis. Let's try to do an analysis of QuickHuskySort's performance. This expands on the $T_1/T_2/T_3$ decomposition introduced in § 3.4 above, replacing its abstract time constants with concrete array-access counts, and extending it to cover RHSort (§ 3.2) as an alternative to quicksort for the T_2 step. The first thing to note is that, like merge sort, QuickHuskySort requires $\mathbf{O}(N)$ extra space. This is because, in addition to the array of object references, we must have an equal-length array of longs (the huskyCode values).

Secondly, QuickHuskySort is not stable unless using the merge sort variation. The derivation of the two comparison figures is a counting argument rather than a measurement, and it rests on one coefficient we

estimated rather than measured; it is set out in § A.1.

Encoding's cost has not simply been assumed: we timed the encoding phase in isolation, separately from the sort that follows it, to confirm it directly. At $N = 1{,}000{,}000$, encoding takes 67.7ms for English words (9.7% of QuickHuskySort's 697.9ms total, 24.8% of RHSort's 272.7ms) and 211.1ms for Chinese names encoded via pinyin (16.9% of QuickHuskySort's 1252.7ms, 27.8% of RHSort's 760.9ms) — confirming that encoding cost, while real and non-negligible, especially for the more elaborate pinyin encoding, remains a minority of total time in every case measured, consistent with the linear-and-subordinate role it is assumed to play above. Encoding's share of the total is larger for RadixHuskySort than for QuickHuskySort in both cases, simply because RHSort's own contribution to the total is smaller — as the sort phase gets cheaper, whatever cost remains in the encoding phase necessarily becomes a proportionally larger slice of what is left, which suggests encoding cost (particularly for pinyin) is the more promising target for further optimization now that the sort phase itself is no longer the dominant cost. The consequence of this is that, as expected, the linear phase of QuickHuskySort is relatively high for small N but that is soon swamped by the saving of comparisons in the linearithmic phase. The break-even point is actually a very small value of N: four. As N increases, we expect the advantage of QuickHuskySort to increase further. This is generally borne out by the benchmarks. Please refer to table 5.1.

The same style of analysis extends naturally to RadixHuskySort (§ 3.2). Each of its $\frac{64}{b}$ digit passes touches every element a small constant number of times, c, to bucket it and to record its new position in the (deferred) index array — with no partitioning or pivot-comparison overhead, and critically, no logarithmic term at all. Taking $c = 4$ (the same per-exchange cost used for QuickHuskySort's swaps above), adding the final $\mathbf{O}(N)$ pass that applies the resulting permutation to the object array (2 array accesses per element), and the same encoding cost ($(k + 1)N$) and cleanup-pass cost ($(2 + j + 4)pN$, unaffected by which algorithm sorts the codes in step 2), the total array accesses for RadixHuskySort (A_r) come to:

$$A_r = \left(4 \cdot \frac{64}{b} + 2 + k + 1\right) N + (2 + j + 4)pN$$

Using the same illustrative constants as above ($j = 4$, $k = 7$, $p = 0.1$) and $b = 16$ (four digit passes, near the middle of the digit-width plateau found empirically — see § 6.3):

$$A_r = 27N$$

Table 5.1 extends the earlier comparison with this term. At $N = 1{,}000{,}000$, $A_r \approx 27{,}000{,}000$ against $A_h \approx 97{,}000{,}000$ and $A_m \approx 161{,}000{,}000$ — a predicted advantage of roughly 3.6x over QuickHuskySort, consistent with the 2-4x speedups actually measured (§ 6.3). The model does predict one caveat: equating A_r and A_h gives a theoretical break-even around $N \approx 17$, somewhat higher than QuickHuskySort's own break-even against merge sort ($N = 4$, above) — because RHSort's per-pass cost has no logarithmic term to shrink relative to at very small N. This crossover is far below every array size actually tested, so it has no practical effect on the results reported here, but it is worth stating rather than leaving the model looking like radix wins unconditionally at every N.

Since the final pass of HuskySort will be cleaning up a relatively small number of inversions, does it matter whether we use insertion sort or Timsort? Below about 50,000 Strings it makes barely any difference, and insertion sort is sometimes faster; beyond that Timsort wins out, by a factor of four and a half at $N = 4{,}000{,}000$. That measurement, and the choice it settles, are in § A.5.

6 Results and Observations.

6.1 Benchmarks. Table 6.1 reports absolute sort times for every data type and corpus. The two radix columns bracket the digit-width plateau of § 6.3. Throughout, *system sort* means *Arrays.sort*, the JVM's default for objects, and every sorter in the table is single-threaded. At the larger sizes a practitioner who cared about speed would reach instead for *Arrays.parallelSort*, so the serial comparison is the conservative one to draw against RadixHuskySort and the generous one to draw against a parallel sorter; § A.7 draws the parallel-against-parallel comparison, and § A.8 reports what parallelizing our own digit passes does and does not buy. Dual-pivot quicksort and a from-scratch quicksort were measured too; they bear on QuickHuskySort rather than on RHSort, so they are in § A.6.

6.2 Summary. HuskySort's target application is the sorting of randomly ordered arrays of objects on the JVM (Java Virtual Machine). Let's eliminate the non-use-cases first:

- Partially-ordered arrays: that's to say, arrays with only a linear number of inversions as would be the case when adding a relatively small number of records to a large existing index. For such use cases, Timsort (the Java system sort for objects) would be more suitable.
- Arrays of primitives: in Java, a clear distinction exists between primitive types (char, byte, short,

Table 5.1: Theoretical comparisons (array accesses, dimensionless counts)

N	$N \ln N$	merge sort	QuickHuskySort	RHSort
4	6	72	71	108
1,000	6,908	81,439	53,210	27,000
1,000,000	13,815,511	160,878,371	97,419,268	27,000,000
1,000,000,000	20,723,265,837	240,317,557,125	141,628,901,356	27,000,000,000

Table 6.1: Sort times by data type (JMH, ms, mean ± 99.9% CI). Numeric types at $N = 500{,}000$. For Chinese names the system-sort column is a system sort performing the same pinyin ordering; code-point order is a different and much cheaper task (§ 6.3).

Data type	N	System sort	QuickHuskySort	RHSort/8	RHSort/16
Integer	500,000	121.2±0.6	89.7±3.5	31.2±0.5	25.2±0.2
Double	500,000	141.7±1.3	97.8±3.6	34.0±0.5	31.0±0.3
Long	500,000	135.8±1.2	97.7±1.0	30.2±0.2	28.6±0.3
BigInteger	500,000	208.4±1.7	152.3±1.7	56.9±0.3	54.1±0.2
BigDecimal	500,000	262.2±1.6	144.1±2.1	54.1±0.3	52.5±0.3
Dates	20,000	3.793±0.007	—	0.836±0.002	0.789±0.002
English words	32,000	13.69±0.19	9.72±0.12	4.63±0.16	4.18±0.16
	200,000	141.39±2.59	94.74±2.44	55.01±2.20	50.27±2.51
	1,000,000	1120.87±14.43	697.88±16.15	355.26±24.71	272.74±6.46
Chinese words	32,000	9.78±0.08	5.00±0.04	2.10±0.03	1.89±0.02
(code-point order)	200,000	71.34±0.63	30.90±0.13	13.49±0.15	10.85±0.10
	1,000,000	447.12±2.28	219.80±1.99	83.25±2.31	65.46±1.47
Chinese names	32,000	69.1±1.4	26.6±0.3	24.1±0.2	23.1±0.3
(pinyin order)	200,000	526.2±9.5	187.2±1.0	157.0±1.3	154.9±1.2
	1,000,000	3112.9±68.0	1252.7±6.4	775.7±4.2	767.2±14.0
Tuples	20,000	3.72±0.01	2.79±0.02	—	1.30±0.01
	100,000	23.24±0.19	17.65±0.16	—	6.82±0.05
	500,000	209.53±7.98	142.61±1.83	—	57.98±0.76

int, float, long, double) and objects. Dual-pivot quicksort (the Java system sort for primitives) is already the fastest algorithm for sorting such types.

Thus, HuskySort is best for any object array where there's no expectation of partial ordering. Typical improvements over Timsort can be read directly off Table 6.1 above, which gives absolute times with 99.9% confidence intervals rather than summary ranges: 1.4–1.8x across the five numeric types at $N = 500{,}000$, 1.4–1.6x on English words, 2.0–2.3x on Chinese words, and 1.3–1.5x on tuples. Note that the English margin *widens* with array size rather than decaying, from 1.41x at $N = 32{,}000$ to 1.61x at $N = 1{,}000{,}000$.

6.3 Radix Sort Results. We benchmarked every data type and corpus covered above using RadixHuskySort (§ 3.2) in place of Introsort for step 2, using JMH (the Java Microbenchmark Harness, with proper fork isolation and warmup) on JDK 21.

Table 6.2 reports RHSort's advantage over QuickHuskySort as a direct multiplier — e.g., **2.9x** means radix took roughly a third as long — rather than as a percentage, since a percentage figure close to or above 100% becomes ambiguous about exactly which quantity it is a percentage *of*.

This is worth confronting directly, since a reader might reasonably ask whether either advantage erodes as N grows: a constant-factor saving on a linearithmic sort could be swamped at scale. Neither does. QuickHuskySort's margin over the system sort on English text *widens* with size, from 1.41x at $N = 32{,}000$ to 1.49x and then 1.61x (Table 6.1), and holds roughly constant on both Chinese corpora. RHSort's advantage over QuickHuskySort likewise does not shrink with N: at every size and data type in Table 6.2, the margin is 1.5x or greater, consistent with the theoretical model of § 3.2 predicting no logarithmic term at all, so there is no analogous reason to expect the advantage to erode as

Table 6.2: RHSort's advantage over the comparison-based husky sort it replaces, as a direct multiplier. Every row uses RHSort/16, so those that appear in Table 6.1 can be checked by dividing its two husky columns. Eleven-bit digits are a few per cent faster for *Long* and for dates, within the plateau of § 6.3, but a table mixing widths would match nothing printed. The dates baseline partitions three ways rather than on two pivots.

Data Type	N	Advantage over QuickHuskySort
`Dates`	20,000	5.5x
`Integer`	500,000	3.6x
`Long`	500,000	3.4x
`Strings of Chinese words (code-point order)`	1,000,000	3.4x
`Double`	500,000	3.2x
`BigInteger`	500,000	2.8x
`BigDecimal`	500,000	2.7x
`Strings of English words`	1,000,000	2.6x
`Tuples`	500,000	2.5x
`Building permits (composite key)`	198,900	2.1x
`Strings of Chinese names (pinyin order)`	1,000,000	1.6x

N grows further. The spread across that table is itself informative, and follows the three factors of § 3.2. Chinese names sorted by pinyin order are the smallest margin measured (1.6x), because both QuickHuskySort and RadixHuskySort must pay the same cleanup pass for a coding that two or three characters of recurring syllables leave badly imperfect. Dates show the largest margin (5.5x) because that coding is provably perfect and never needs a cleanup pass at all, letting RHSort's linear-time advantage show through with nothing else in the way. The two lie at opposite ends of the same axis: what separates them is not the data type but the perfection of the encoding.

The building permits row is the only one in that table not measured on data we generated: 198,900 real municipal records ordered by a composite key of three fields (§ 5.1). At 2.1x it is tenth of eleven, which is the point worth making about it — it is not our largest margin, but it is our largest on data whose distribution we did not choose. § A.10 gives the case study, and § A.12 what a general composite encoder would have to provide, that one having been written by hand.

The measurements were reproduced qualitatively on machines A and B of § A.3, which differ from the machine of record and from each other in vendor, instruction set, core design and JVM generation. We quote no figures from them, since mixing environments within a comparison would rob it of meaning, but every ordering reported here holds on all three.

6.4 Use-Case Guidance. § 6.2 already eliminates two cases outright: partially-ordered arrays, where Timsort's own adaptivity wins, and arrays of already-cheap-to-compare primitives, where dual-pivot quicksort is already the fastest option. For the remaining case — arrays of objects with no expectation of partial ordering — the decision is governed by the three factors of § 3.2 rather than by the size of the array, and Table 6.3 puts them in the order a practitioner can answer them. Size enters only once those are settled, and then only near the bottom of the range.

Only when all of those are answered does array size matter, and then chiefly at the small end, where each algorithm's fixed setup cost has not yet been amortized. For String keys we measured every crossover directly (JMH, English corpus, $N = 4$ through 10,000). System sort is the fastest of the group up to $N = 20$, though only narrowly there (0.462μs against insertion sort's 0.489μs). A plain, binary-search-based insertion sort wins outright from $N = 50$ through $N = 200$ — the one regime in this paper where plain insertion sort, not either HuskySort variant, is the best available choice. QuickHuskySort takes over between $N = 200$ and $N = 500$, and remains the best choice up to roughly $N = 2{,}000$, since RadixHuskySort's own fixed setup cost is not yet amortized below that point. That cost can be read directly off the measurements: RadixHuskySort's time is a near-constant 166μs floor from $N = 4$ to $N = 500$, the allocation and initialisation of its counting structures, which only the linear term begins to dominate past a few thousand elements. Somewhere between $N = 2{,}000$ and $N = 10{,}000$ it takes over; its advantage then grows and holds at every larger size tested (Table 6.2). Every one of these crossovers shares the same shape: each successive algorithm carries a larger fixed setup cost that pays for itself only once there is work enough to amortize it — the story § A.8 finds one level down, in ParallelRHSort's thread and

Table 6.3: Which sort to reach for. The situations are ordered by how much they decide, and array size settles nothing until all of them are. Every sorter named here is single-threaded; with cores to spare, *Arrays.parallelSort* wins outright on strings and on the permits (§ A.7).

Your situation	Reach for	Evidence
The ordering is already cheap to evaluate — primitives, or a single numeric field	dual-pivot quicksort	cost of encoding cannot be recouped (§ 6.2)
The ordering is composite or expensive, *and* the key packs exactly into 64 bits	RadixHuskySort; no cleanup pass is required	the largest margins we measure: dates 5.5x, building permits 2.1x over QuickHuskySort
The ordering is composite or expensive, but the encoding is imperfect	RadixHuskySort, even allowing for the cleanup pass	that pass costs a sixth to a quarter past $N = 100{,}000$, even with nothing to correct (§ 3.4)
A sort specialised to your type already exists	use it, if it covers your data	on English words a tuned MSD radix sort beats us, though our MSD indexes extended ASCII only and cannot sort either Chinese corpus (§ 3.2)
The data defeats the encoding's capture window — long shared prefixes, or keys wider than the coder can see	System sort — *Arrays.parallelSort* if cores are free	every husky variant loses its advantage (§ A.13)
None of the above settles it, or the keys are strings	by array size (§ 6.4)	each sort's fixed setup cost is repaid at a different N

barrier setup. They were measured for String keys only. The costs involved are architectural rather than type-specific, so other key types plausibly show similar curves at similar sizes, though we have not confirmed it.

Two further caveats from elsewhere in this paper complete the picture. If the source data defeats the husky encoding's fixed capture window entirely — a shared prefix longer than the encoding's own character limit, for instance — neither QuickHuskySort nor RadixHuskySort helps regardless of N (§ A.13): both lose to plain System sort, since the encoding pass becomes pure waste and the mandatory cleanup pass must do all the real work anyway. And spare cores change the answer, which we now measure rather than concede (§ A.7). *Arrays.parallelSort* is one method call and no code of ours, and on sixteen cores it beats serial RadixHuskySort by 2.32x on a million English strings and 2.15x on the permits. Parallel against parallel the answer depends on what a comparison costs: *ParallelRadixHuskySort* leads by 1.40x at ten million *Long*s, trails by 1.09x at two million, and on the permits leads at every size by 1.25x to 1.59x. Where cores are free and the JDK can compare the key cheaply, reach for *Arrays.parallelSort*; where the comparison is expensive — a composite key, or a collation — our parallel variant is now the faster of the two, and our serial one still earns its place on arrays whose ordering the encoding replaces.

7 Conclusion. The original idea behind HuskySort — QuickHuskySort — was to shift work from the linearithmic phase of sorting into the linear ones, prelude and postlude, reducing array accesses and so processing time. This has been demonstrated such that, when sorting objects rather than primitives, HuskySort is faster than dual-pivot quicksort wherever the ordering is expensive to evaluate — by 1.4x on *BigInteger* and 1.7x on *BigDecimal* (§ A.6). On *Integer*, *Double* and *Long*, where a comparison costs almost nothing, the encoding has nothing to amortise and dual-pivot quicksort is ahead instead, by up to a tenth. It is faster than Timsort for arrays which are not partially ordered, above a few hundred elements; below that, Timsort's own tuned handling of small arrays wins instead. It is especially fast for types whose ordering is composite or expensive to evaluate and whose keys encode perfectly, where the encoding replaces the whole comparison and no cleanup pass is needed. Strings are not reliably that case: the one domain with a mature specialised literature of its own

(§ 3.2), and the most variable, spanning nearly the whole range of our results — including our narrowest margin, Chinese personal names in pinyin order.

For RadixHuskySort the improvement is larger still above a few thousand elements, and since that variant has no linearithmic phase at all, the saving comes from bit manipulation replacing calls to *equals*.

These findings are not an artifact of a single machine: the same qualitative pattern held on three environments spanning two CPU vendors and three architectures (§ A.3), the last a 16-core ARM instance with no performance/efficiency split to confound it. That run also settled a question the parallel variant had left open (§ A.8): its scaling ceiling past a handful of threads is a property of the design — the sequential histogram-combine once per pass — not of any machine's mix of core types.

HuskySort's advantage is specific to a kind of workload rather than universal, and needs all three of the following: an array of at least a few hundred elements, growing to a few thousand before RadixHuskySort's per-pass setup cost is repaid (§ 6.4); no expectation of pre-existing order, which favours Timsort's adaptivity instead (§ 6.2); and a comparison costing more than a machine-word check — strings, arbitrary-precision numbers and multi-field records, as opposed to *int*, *long* or *double*, where dual-pivot quicksort wins outright.

The advantage is no longer only a serial one, but it is conditional. A proxy-key sort has four phases: the encoding, the digit passes, the permutation, and the cleanup. Earlier versions of this work parallelized only the digit passes, and reported — correctly for that implementation — that the parallel variant was a negative result. With the encoding parallelized as well, and the cleanup available in an opt-in parallel form, three of the four phases divide, and the comparison against *Arrays.parallelSort* turns on the corpus (§ A.7). We lead on the permits at all three sizes, by 1.25x to 1.59x; on *chinesenames* by 5.74x to 7.17x, where the baseline must pay pinyin collation on every comparison; on *chinese* by 1.10x to 1.49x once the cleanup runs in parallel; and at ten million *Long*s by 1.40x, which is the cheapest comparison we measure and so the case where husky coding has least to offer. We trail at two million *Long*s, by 1.09x. On *english* the measurements did not converge, and we make no claim there in either direction. The conditionality is the finding rather than a caveat on it: what decides each cell is how much disorder the coder leaves for the cleanup, not the size of the array, which is why two corpora of the same length can want opposite answers.

One phase still does not divide, and one expectation of ours was wrong. The permutation remains sequential: it is an independent scatter under a bijection, so nothing about it resists parallelization, but we have not done it. The cleanup, which we had expected to be the hard case because the elements are not independent, in fact parallelizes — the dependence lives in the merge, and merging divides. What it is *not* is uniformly worth doing. Enabling it moves *chinese* at 200,000 from 0.64x to 1.10x against the baseline and *chinese* at a million from 1.15x to 1.49x, but leaves *chinesenames* unchanged to within its intervals at all three sizes, because that corpus's cleanup is already the overwhelming majority of a sort the baseline loses badly in any case. Where the cleanup is cheap, dividing it cannot repay the division: a serial Timsort on a nearly ordered array finds long runs and stops, while a parallel merge still pays its merge levels. That is why the parallel cleanup is opt-in and why we encode no rule for choosing it: the quantity that decides is residual disorder, and N alone does not express it. Settling that rule, and parallelizing the permutation and repairing the sequential histogram-combine of § A.8, is the clearest future work here. The honest summary for a practitioner is narrower than either of the claims this paper has made in earlier drafts: where comparisons are expensive — the permits, and Chinese names under pinyin — a parallel proxy-key sort is now the faster choice on this machine, and where they are cheap the question remains open.

Acknowledgments To Darshan Dedhia, Akshay Bhusare, Kartik Kumar for their help with this project.

References

[1] A. Aggarwal and J. S. Vitter, *The input/output complexity of sorting and related problems*, Commun. ACM, 31 (1988), pp. 1116–1127, https://doi.org/10.1145/48529.48535.

[2] G. Antoshenkov, D. Lomet, and J. Murray, *Order preserving string compression*, in Proceedings of the Twelfth International Conference on Data Engineering, IEEE Computer Society Press, 1996, pp. 655–663, https://doi.org/10.1109/ICDE.1996.492216.

[3] N. Auger, V. Jugé, C. Nicaud, and C. Pivoteau, *On the worst-case complexity of timsort*, CoRR, abs/1805.08612 (2018), http://arxiv.org/abs/1805.08612, https://arxiv.org/abs/1805.08612.

[4] J. L. Bentley and M. D. McIlroy, *Engineering a sort function*, Software: Practice and Experience, 23 (1993), pp. 1249–1265, https://doi.org/10.1002/spe.4380231105.

[5] J. L. Bentley and R. Sedgewick, *Fast algorithms for sorting and searching strings*, in Proceedings of the Eighth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA '97, USA, 1997, Society for Industrial and Applied Mathematics, pp. 360–369.

[6] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, MIT Press, Cambridge, Massachusetts, fourth ed., 2022. Counting sort is Section 8.2; the radix sort built on it is Section 8.3.

[7] DataSF, *Building permits*. San Francisco open data portal, dataset `i98e-djp9`, https://data.sf.gov/Housing-and-Buildings/Building-Permits/i98e-djp9/about_data, 2018. Extract declared as covering 2013-01-01 to 2018-02-25, obtained from a public redistribution on Kaggle (https://www.kaggle.com/datasets/aparnashastry/building-permit-applications-data) licensed under the Open Data Commons Database Contents License (DbCL) v1.0, https://opendatacommons.org/licenses/dbcl/1-0/.

[8] M. Frigo, C. E. Leiserson, H. Prokop, and S. Ramachandran, *Cache-oblivious algorithms*, in 40th Annual Symposium on Foundations of Computer Science (Cat. No.99CB37039), 1999, pp. 285–297, https://doi.org/10.1109/SFFCS.1999.814600.

[9] D. Goldhahn, T. Eckart, and U. Quasthoff, *Building large monolingual dictionaries at the leipzig corpora collection: From 100 to 200 languages*, in Proceedings of the Eight International Conference on Language Resources and Evaluation (LREC'12), N. C. C. Chair), K. Choukri, T. Declerck, M. U. Doğan, B. Maegaard, J. Mariani, A. Moreno, J. Odijk, and S. Piperidis, eds., Istanbul, Turkey, 2012, European Language Resources Association (ELRA). 23-25.

[10] R. C. Hillyard, Y. Liaozheng, and S. V. K R, *Huskysort*, CoRR, abs/2012.00866 (2020), https://arxiv.org/abs/2012.00866, https://arxiv.org/abs/2012.00866.

[11] C. A. R. Hoare, *Algorithm 64: Quicksort*, Commun. ACM, 4 (1961), p. 321, https://doi.org/10.1145/366622.366644.

[12] V. Jugé, *Adaptive shivers sort: An alternative sorting algorithm*, in Proceedings of the Thirty-First Annual ACM-SIAM Symposium on Discrete Algorithms, SODA '20, USA, 2020, Society for Industrial and Applied Mathematics, pp. 1639–1654, https://doi.org/10.1137/1.9781611975994.101.

[13] A. LaMarca and R. E. Ladner, *The influence of caches on the performance of sorting*, Journal of Algorithms, 31 (1999), pp. 66–104, https://www.sciencedirect.com/science/article/pii/S0196677498909853.

[14] A. López-Ortiz, M. Mirzazadeh, M. A. Safari, and H. Sheikhattar, *Fast string sorting using order-preserving compression*, ACM Journal of Experimental Algorithmics, 10 (2005), https://doi.org/10.1145/1064546.1180611.

[15] P. McIlroy, *Optimistic sorting and information theoretic complexity*, in Proceedings of the Fourth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA '93, USA, 1993, Society for Industrial and Applied Mathematics, pp. 467–474.

[16] D. R. Musser, *Introspective sorting and selection algorithms*, Software: Practice and Experience, 27 (1997), pp. 983–993, https://doi.org/10.1002/(SICI)1097-024X(199708)27:8<983::AID-SPE117>3.0.CO;2-#, https://arxiv.org/abs/https://onlinelibrary.wiley.com/doi/pdf/10.1002/.

[17] T. Peters, *Timsort - Python*, 2002, https://svn.python.org/projects/python/trunk/Objects/listsort.txt. Retrieved November 3, 2020.

[18] R. Sinha and J. Zobel, *Cache-conscious sorting of large sets of strings with dynamic tries*, ACM J. Exp. Algorithmics, 9 (2004), https://doi.org/10.1145/1064546.1180622.

[19] Wainshine, *Chinese-names-corpus*. GitHub repository, commit `95b1a18`, 2020, https://github.com/wainshine/Chinese-Names-Corpus.

[20] S. Wild and M. E. Nebel, *Average case analysis of java 7's dual pivot quicksort*, in Algorithms — ESA 2012, Springer Berlin Heidelberg, 2012, https://doi.org/10.1007/978-3-642-33090-2_71.

[21] V. Yaroslavskiy, *Dual Pivot Quicksort*, 2009, https://codeblab.com/wp-content/uploads/2009/09/DualPivotQuicksort.pdf. Retrieved November 3, 2020.

[22] H. Zhang, B. Meng, and Y. Liang, *Sort race*, CoRR, abs/1609.04471 (2016), https://arxiv.org/abs/1609.04471.

A Appendix.

A.1 The array-access analysis of QuickHuskySort. § 5.2 counts array accesses for RHSort and compares the result with QuickHuskySort and merge sort in Table 5.1. This section derives the two comparison figures.

It is a counting argument, and one step in it is an estimate rather than a count: the factor of 0.6 below, which stands for the proportion of object-reference swaps whose targets have not already been fetched into cache. We have not measured it. Everything else here is arithmetic over the operation counts of the algorithms concerned, and the RHSort figure in § 5.2 needs no such factor — $4 \cdot 64/b$ digit passes is a count, not an estimate. The comparison should be read as an order-of-magnitude argument, which is all the paper asks of it; the measurements of § 6.3 are what carry the result.

How many comparisons and swaps does QuickHuskySort require, on average? Wild and Nebel's average-case analysis of dual-pivot quicksort [20] gives, approximately:

- Comparisons: $1.9N \ln N$;
- Swaps: $0.6N \ln N$.

A comparison reads two array elements, though because one comparand is always the pivot — already in hand — the effective count is closer to one. Each swap requires 4 array accesses, although again, we will be swapping elements that have already been accessed. The effective number of (normalized) array accesses is, in total, approximately 4 rather than the expected 3.8 + 2.4. That is to say, the coefficient of $N \ln N$ is approximately 4. Wild and Nebel's is the right analysis to apply here: step 2's partitioning is itself dual-pivot (§ 3.1). [1] However, in QuickHuskySort, each swap also must swap the object references. These will not, in general, have been pre-fetched and so the effective total normalized array accesses (A) is approximately:

$$A = 4 + 0.6 * 4 = 6.4$$

Additionally, QuickHuskySort requires N huskyCode constructions. Each of these requires $k + 1$ array accesses where k is proportional to the number of fields contributing to the huskyCode. For a string, that will be the number of characters in the string (except that this is limited to a relatively small number according to the encoding). For ASCII strings, it is nine. For Unicode strings, it is four.

[1] Note that, since dual-pivot quicksort is not actually implemented in the Java library for Comparable objects, we re-implemented the class for objects.

However, keep in mind that this is linear and, while it can't be completely ignored, it doesn't contribute to the overall growth of the complexity.

Once the sort of the codes is complete, we will assume that there are pN remaining inversions, where p is the probability of an idiogenic (self-inflicted) inversion. Again, while we should not ignore it completely, it is linear. The number of array access for the comparisons/swaps to fix these will be $(2 + j + 4)pN$, where j is the number of fields to be compared to get a result (j is usually smaller than k).

So, the total number of array accesses (A) for husky-sorting an array of length N is:

$$A = (6.4 \ln N + (2 + j + 4)p + k + 1)N$$

Timsort would be hard to analyze in the general case and so we instead analyze merge sort, which should be approximately the same number of array accesses in the average case.

For merge sort, the analysis is much simpler: the number of comparisons for a randomly ordered array of length N is $N \log_2 N$. Merge sort does not use swaps, *per se*, but instead uses copies. Assuming that we optimize the extra copying, there will be N copies to get the algorithm started and N per level. Thus, the number of copies will be $N \log_2 N$. Each of these copies requires two array accesses while each comparison requires 2 + j (described above) array accesses.

The total number of array accesses (A) for merge sort is, therefore:

$$A = (4 + j)N \log_2 N + 2N$$

Given that $\log_2 N = 1.44 \ln N$, and assuming that:
$j = 4$ corresponding to the comparison of two strings that start with the same character but differ in the second character; and
$k = 7$ and $p = 0.1$; then
the totals (A_m for merge sort and A_h for QuickHuskySort) come to:

$$A_m = 11.5N \ln N + 2N$$

$$A_h = 6.4N \ln N + 9N$$

A.2 Prior comparison-based sorting algorithms. The algorithms reviewed in § 2, with the standard complexity and stability characteristics referred to there.

A.3 Benchmark environments. Machine C is the machine of record and the source of every figure quoted in this paper. Machines A and B appear only as the qualitative cross-checks of § 6.3, and machine A additionally supplies the cleanup-sort comparison of § A.5.

Table A.1: Prior comparison-based sorting algorithms

Algorithm	Average case	Worst case	Stable	Notes
Insertion sort	$\mathbf{O}(N + I)$	$\mathbf{O}(N^2)$	Yes	I = number of inversions; fast only for small or nearly-sorted N
Merge sort	$\mathbf{O}(N \log N)$	$\mathbf{O}(N \log N)$	Yes	Linearithmic regardless of input order; $\mathbf{O}(N)$ extra space
Quicksort	$\mathbf{O}(N \log N)$	$\mathbf{O}(N^2)$	No	In-place, cache-friendly; the quadratic worst case is mitigated in practice (e.g., Introsort's heapsort fallback [16])
Timsort	$\mathbf{O}(N + I)$ to $\mathbf{O}(N \log N)$	$\mathbf{O}(N \log N)$	Yes	Hybrid of merge sort and insertion sort, adaptive to existing ascending/descending runs; more precisely $\mathbf{O}(N + N \log r)$ for r runs [3]

Table A.2: The three benchmark environments. Machine A supplies the comparison-sort results of § 5.2; machine B the radix-sort and JMH development work; machine C, the machine of record, every figure quoted in this paper. Machine C's cache geometry is not exposed to the guest, hence the dash; machine B's is given per core cluster, since its two clusters differ and the benchmarks run on the performance cores.

	A — comparison sorts	**B** — development	**C** — machine of record
Model	MacBook Pro (MacBookPro14,3)	MacBook (Apple Silicon)	AWS EC2 c7g.4xlarge
Processor	Quad-core Intel Core i7, 2.8 GHz, Hyper-Threading	Apple M1, 8 cores (4 performance + 4 efficiency)	ARM Neoverse V1 (Graviton3), 16 vCPU, 2.6 GHz
Cache	256 KB L2 per core, 6 MB L3	Performance cores: 192 KB L1I / 128 KB L1D each, 12 MB shared L2. Efficiency cores: 128 KB / 64 KB, 4 MB shared L2. 128 B line	—
Memory	16 GB 2133 MHz LPDDR3	16 GB unified	30 GiB
OS	—	macOS 26.5.2	Amazon Linux 2023
JVM	1.8.0_152	OpenJDK 21.0.10	OpenJDK 21.0.12 (Corretto)

A.4 Discussion of coding accuracy. It is not essential that coding is perfect for HuskySort to work well. We have experimented with deliberately throwing off the encoding quite drastically. If the probability of one of these deliberate mis-codings is less than approximately 25 percent, HuskySort can still save time. The inference that we can take from this is that it is not critical that encodings are all very good. We can be confident that we are still saving time up to about one in four errors.

These deliberate errors were tested for Bytes and Integers but not the other generic types.

A.5 Timsort or insertion sort for the cleanup pass. Step 3 needs a stable sort for an array that is nearly ordered already, and both Timsort and insertion sort qualify. Below about 50,000 elements the choice barely matters and insertion sort is sometimes marginally faster; beyond it Timsort wins decisively, by a factor of four and a half at N = 4,000,000. That is why step 3 uses the system sort. These figures are from machine A of § A.3 and are the paper's only measurements not taken on the machine of record; they are reported here because the effect is an order of magnitude larger than any plausible machine difference, and because the choice they settle is a design decision rather than a result.

A.6 Comparison-sort baselines for the numeric and tuple types. Table 6.1 omits two baselines that were measured alongside the others: dual-pivot quicksort adapted to objects, and the JDK's own primitive-array sort applied to unboxed values. Both bear on QuickHuskySort rather than on RHSort, which is why they are here rather than in the body. The comparison they support is the one drawn in § 7: husky

Table A.3: Timsort or insertion sort as the cleanup pass (time in milliseconds)

N	QuickHuskySort, Timsort cleanup	QuickHuskySort, insertion-sort cleanup
1000	0.14	0.14
2000	0.31	0.30
4000	0.63	0.64
8000	1.46	1.56
16000	3.24	3.19
32000	6.56	6.48
64000	16.85	19.41
125000	47.74	54.90
250000	168.21	167.90
500000	259.37	373.43
1000000	602.08	1202.61
2000000	1234.90	3129.23
4000000	2423.62	11018.19

encoding repays itself when the ordering is expensive, and not otherwise.

A.7 *Arrays.parallelSort* as a parallel baseline. Every sorter in Table 6.1 is single-threaded, and at the sizes reported there a practitioner who cared about speed would not call *Arrays.sort* at all. They would call *Arrays.parallelSort*, which is one method call away and needs no code of ours. This section reports that comparison on the machine of record. It is the comparison a practitioner faces, and both sides now draw on the same pool: *Arrays.parallelSort* uses the common *ForkJoinPool* — fifteen workers plus the calling thread on this machine's sixteen cores — and the *pAll* configuration of *ParallelRadixHuskySort* reported here uses the same parallelism rather than a fixed four or eight threads. Two variants of ours appear: *pAll* with the sequential cleanup, which is the default, and *pAll* with the opt-in parallel cleanup, in which three of the four phases run in parallel and only the permutation is sequential. The opt-in form is not the default because what it gains depends on how much disorder the coder left, which varies by corpus and not by size alone (§ A.9).

On strings the answer depends on the corpus and on whether the cleanup runs in parallel (Table A.5). With the default sequential cleanup we lead on *chinesenames* at every size and on *chinese* at a million, tie at *chinese* 32,000, and lose at *chinese* 200,000. With the parallel cleanup enabled we lead on every *chinese* and *chinesenames* cell, by 1.10x to 1.49x on *chinese* and by 5.74x to 7.17x on *chinesenames*. The *english* rows moved the same way — the parallel cleanup takes 200,000 from 24.05 to 9.84 against a baseline of 11.43 — but *english* did not converge within the measurement window, so we make no *english* claim here and mark the rows accordingly.

The spread between the corpora is the coder's doing rather than the sorter's, and it is the same effect § A.11 describes: the radix phase cuts *chinesenames*' run count by only 3.1x where it cuts *english*'s by 62x, so *chinesenames* arrives at the cleanup with far more work left to divide. That is why it gains most from a parallel cleanup in absolute terms and why it stands furthest ahead of a comparison sort that must pay the pinyin collation on every comparison.

On *Long[]* the answer depends on the size, and the crossover falls between the two we measured. At $N = 2{,}000{,}000$ the baseline is ahead: 90.4±3.1 against our best row's 98.8±1.4, so eight threads is 1.09x *slower*. At $N = 10{,}000{,}000$ the order reverses and eight threads leads by 1.40x, 561.5±19.8 against 785.4±10.4, with disjoint intervals. This is the cheapest comparison we measure, and so the case where husky coding has least to offer; that the parallel form still leads at ten million is therefore a result about the digit passes rather than about the coding.

The permits are the most instructive case, because the mechanism is legible and because an earlier version of this paper reported them as the least flattering one. They are no longer that. The parallel form now leads the baseline at all three sizes — by 1.57x at $N = 32{,}000$, 1.25x at 100,000 and 1.59x at the full corpus — and beats its own serial form at the two larger sizes, by 2.88x and 3.41x. At $N = 32{,}000$ serial RHSort/16 remains the fastest thing here, 2.72±0.03 against the parallel form's 3.02±0.06, which is the chunking rule showing through: passes are split into $\max(1, \min(\mathit{parallelism}, N/2^{14}))$ chunks, so at that size there is a single chunk and the parallel form pays its thread-pool construction — which sits inside the

Table A.4: Two further comparison sorts, measured on the same arrays as Table 6.1 and directly comparable with its System sort and QuickHuskySort columns (JMH, ms, mean ± 99.9% CI; numeric types at $N = 500{,}000$). *Dual-pivot quicksort* is Yaroslavskiy's algorithm transcribed for objects, the JDK applying it only to primitives; *quicksort (raw)* unboxes each element to a primitive and sorts that array with the JDK's primitive sort, discarding the objects rather than permuting them — no payload to carry, no encoding abstraction and no cleanup pass — and so bounds from below what any husky variant could achieve on these types. It was measured for the numeric types only; the final block is the Permit tuple.

Data type	N	Dual-pivot quicksort	quicksort (raw)
Integer	500,000	82.5±0.2	38.1±0.1
Double	500,000	89.6±0.5	42.6±0.2
Long	500,000	89.2±0.6	38.2±0.3
BigInteger	500,000	209.9±2.0	42.2±0.1
BigDecimal	500,000	246.2±3.7	47.6±0.0
Tuple	20,000	3.41±0.09	—
	100,000	21.36±0.48	—
	500,000	142.70±4.98	—

Table A.5: Parallel against parallel on the string corpora (ms, mean ± 99.9% CI, all cores). *Ratio* is *Arrays.parallelSort* divided by the better of our two columns, so above 1.00 we lead. The *chinesenames* baseline sorts by pinyin, as ours does, so the two sides do the same job. ‡ marks a row whose first five measurement iterations exceed its last five by more than 10% in some fork, i.e. one that had not converged; no *english* claim is made against this baseline for that reason.

Corpus	*N*	*Arrays.parallelSort*	*pAll*	*pAll*, par. cleanup	Ratio
english	32,000	5.00±0.05	4.71±0.42	4.35±0.35‡	—
english	200,000	11.43±0.21	24.05±1.28‡	9.84±0.13	—
english	1,000,000	89.65±1.61	101.16±18.24‡	54.81±6.50‡	—
chinese	32,000	3.27±0.02	3.27±0.18	2.23±0.02	1.47x
chinese	200,000	6.65±0.03	10.32±0.09	6.03±0.04	1.10x
chinese	1,000,000	46.72±2.27	40.66±0.97	31.32±0.58	1.49x
chinesenames	32,000	21.74±0.12	3.03±0.07	3.04±0.06	7.17x
chinesenames	200,000	46.82±0.84	8.19±0.05	8.16±0.05	5.74x
chinesenames	1,000,000	241.98±7.35	35.17±1.07	34.61±1.04	6.99x

Table A.6: *ParallelRadixHuskySort* against *Arrays.parallelSort* on *Long[]* (ms, mean ± 99.9% CI, 11-bit digits). All columns are from one run. ‡ marks an unconverged row, as in Table A.5.

N	Serial	1 thread	2 threads	4 threads	8 threads	*Arrays.parallelSort*
2,000,000	157.2±4.6‡	198.3±4.6	151.8±4.7	110.9±2.5	98.8±1.4	90.4±3.1‡
10,000,000	960.3±23.9	1104.1±24.2	813.1±24.6	625.8±24.0	561.5±19.8	785.4±10.4

Table A.7: The permits under both parallel sorts (ms, mean ± 99.9% CI). RHSort/16 is serial. *pAll* uses every core and the automatic digit width.

N	System sort	*Arrays.parallelSort*	RHSort/16	*pAll*	8 threads, 16-bit
32,000	14.20±0.21	4.75±0.04	2.72±0.03	3.02±0.06	3.45±0.08
100,000	65.21±0.74	6.26±0.09	14.40±0.56	5.00±0.11	7.04±0.15
198,900	159.08±2.01	13.83±0.13	29.77±0.33	8.72±0.18	11.91±0.19

timed region — for no division of labour. What changed between the two readings is the digit width rather than the thread count. At a fixed 16 bits each chunk builds a 65,536-entry histogram per pass, which at these sizes is more buckets than elements, so bookkeeping scales with chunks times buckets rather than with N; the automatic width avoids that, and the two columns separate accordingly at the full corpus, 8.72±0.18 against 11.91±0.19. That the same fixed width costs nothing on the two-to-ten-million *Long[]* arrays, where each chunk holds hundreds of thousands of elements, is consistent with the same reading. We have not isolated the contributions of pool construction and histogram size separately, so the split between them is a reading of the design rather than a measurement of it. The serial result is unaffected throughout: the system sort remains 5.34x slower than serial RHSort/16 at the full corpus.

A.8 Parallel Radix Sort. Radix sort's digit passes are natural candidates for parallelization (§ 3.2 above). We implemented and benchmarked a parallel variant: each pass is split into a configurable number of contiguous chunks, one per worker thread. Every chunk first computes its own local per-bucket histogram independently, with no synchronization required; a short sequential step then combines those histograms into an exact starting offset, in the output array, for every (chunk, bucket) pair — preserving the stability that step 3's cleanup pass, when needed, relies on, since a chunk's elements always land after all lower-numbered chunks' elements sharing the same bucket; finally, every chunk scatters its own elements into the output arrays independently, using only its own precomputed offsets, so no two chunks ever write to the same location.

An initial implementation dispatched the two phases above to a thread pool separately on every pass. Measurement showed this repeated dispatch itself contributing a real, non-trivial share of the total cost, particularly at more moderate array sizes. The overhead was substantially reduced by starting the worker threads once per sort, rather than once per phase per pass: each thread runs its own loop over every pass, and the two phases synchronize via reused barriers whose associated actions — code that runs exactly once per synchronization point, in whichever thread arrives last — perform the histogram-combine and buffer-swap steps. Table A.8 reports the result (*Long[]*, 11-bit digits, milliseconds, mean ± 99.9% CI).

The comparison here is against RadixHuskySort's own serial implementation and against itself at other thread counts; § A.7 sets it against *Arrays.parallelSort* on the same machine, which is the comparison a practitioner actually faces. Isolating parallelism itself, that's to say comparing 1 thread against 4, both paying identical thread-coordination overhead, gives a statistically resolved speedup at both sizes: 1.39x at $N = 2{,}000{,}000$ and 1.29x at $N = 10{,}000{,}000$, with non-overlapping confidence intervals in both cases. Against the serial implementation the improvement is resolved at both sizes too, 172.0ms to 118.5ms at $N = 2{,}000{,}000$ and 935.2ms to 687.9ms at $N = 10{,}000{,}000$. Unlike on machine B's eight cores, scaling does not stop at four threads: on sixteen uniform cores, eight threads beats four at both sizes, reaching 1.51x and 1.37x over a single thread. It is resolved at $N = 2{,}000{,}000$; at $N = 10{,}000{,}000$ the intervals just touch, so that size shows the direction rather than settling it.

That is still well short of linear, and the shortfall is the more interesting number: an eightfold increase in threads buys between 1.37x and 1.51x, on hardware with no performance/efficiency asymmetry to account for it. Two effects fit, and they are not equally benign. Each pass scatters across the whole array, so the sort may be limited by memory throughput, which extra cores do not supply: a property of the machine. The other belongs to the design — the histogram-combine between a pass's two phases is sequential and costs a step per bucket per chunk, so its work *grows* with the thread count while the parallel work shrinks. That has a consequence the first lacks: the design should fail wherever buckets times threads approaches N, and § A.7 finds exactly that on the permits.

A.9 The cleanup pass as N grows. § 3.4 isolates the cleanup pass at $p = 0$ on the municipal records. The figures are these.

That comparison costs 16.1 ns at $N = 32{,}000$, 60.3 ns at $N = 100{,}000$ and 74.2 ns at $N = 198{,}900$ — 7.3%, 17.2% and 17.3% of the running time of the sort that contains it — with non-overlapping 99.9% confidence intervals throughout. Three further runs, one of them on different hardware, agree on the shape at different levels; across all four the pass costs between a sixteenth and a quarter of the total, and between a sixth and a quarter at the two larger sizes. The percentages are the less stable figures, being ratios of two quantities that both move. The shape is a threshold rather than a trend — a rise by a factor of roughly three to four between the first two sizes, then a levelling off — and it falls where the records stop fitting in cache. That last reading is inference, but the magnitudes support it: the system sort compares these same records 3.25 million times in 151 ms, or 46 ns each, so the cleanup pass at 74 ns costs about 1.6 times as much per comparison using the same comparator. Step 2 permutes references without moving the records, so neither sort walks the heap in order — but the system sort touches each record

Table A.8: Parallel RHSort: serial vs. 1, 2, 4, and 8 worker threads

N	Serial	1 thread	2 threads	4 threads	8 threads
2,000,000	172.0±5.7	165.2±2.2	140.3±2.0	118.5±2.4	109.3±1.8
10,000,000	935.2±36.9	884.5±21.6	731.6±19.9	687.9±22.1	645.3±20.9

some thirty times as a comparison operand, where the cleanup pass touches it twice. A cache miss is amortised over the first and paid in full by the second.

The share therefore *rises* with N, from 7.3% to 17.3%. One objection should be met head on: the pass is linear where the sort containing it is linearithmic, so its share is bound to fall as $1/\log N$.

Both are true, because k_3 is not settled over this range. It grows more than fourfold between the smallest and largest sizes measured while $\ln N$ grows by a fifth, so the cache effect swamps the logarithm completely. Holding k_3 at the value it reaches once the records no longer fit in cache, and growing the rest as $\ln N$, puts the share at some 16% at $N = 10^6$ and 14% at $N = 10^7$: the decline is real but logarithmic, and the pass does not become negligible at any size we expect to meet.

That last calculation is a model rather than a measurement. The sort's own per-element cost also grows faster than $\ln N$ across these three sizes, and the corpus holds 198,900 records, so the measurement cannot be extended on real data.

A.10 Building permits: the case study in full. The permits row of Table 6.2 is the case this mechanism is designed for. The ordering is expensive, since a comparison may have to examine all three fields of the key in turn, and the key nevertheless packs exactly into 60 bits, so no cleanup pass is required at all. Algorithm 3 gives the coder. The date field is the one that depends on the corpus rather than on the schema: eleven bits admit 2,048 consecutive days and the extract spans 1,878, so the coder checks each date against that window and refuses one outside it rather than encoding it wrongly — which is what makes *perfect()* a verified property here rather than an assumption. The corpus also shows why real data is worth the trouble of obtaining: its published window begins on 1 January 2013 and ends on 25 February 2018, but no permit bears either date, nor any other weekend or public holiday — all 198,900 were filed on a business day. A generated corpus would not have that structure unless someone had thought to put it there. Against the system sort, RadixHuskySort is faster by 4.73x, 4.26x and 4.53x at $N = 32{,}000$, 100,000 and 198,900; against QuickHuskySort, by 2.30x, 2.06x and 2.12x.

At 2.1x over QuickHuskySort the row is tenth of eleven. A composite key of three fields is more expensive to compare than a single *Long* but a good deal cheaper than a pinyin lookup per character, so the margin falls squarely inside the range the three factors of § 3.2 predict for it. That is the point worth making about it: it is not our largest margin, but it is our largest on data whose distribution we did not choose, and a mechanism that only worked on synthetic inputs would not be worth reporting.

The same records also supply the cleanup-pass measurement of § 3.4, because the encoding for them is perfect and can be declared so — which is what allows the pass to be switched on and off over identical codes.

There is one comparison this case study loses, and it should be met here rather than left to § A.7 alone. Given sixteen cores, *Arrays.parallelSort* sorts the full corpus in 13.1 ms against serial RHSort/16's 30.9 — 2.35x faster, and faster at that size than either parallel husky configuration. Our own sort leads it at $N = 32{,}000$ by 1.69x, so the crossover falls between 32,000 and 100,000; and since the corpus stops at 198,900 records, real data cannot say whether a second crossover lies beyond it. Two things follow, and they should not be run together. The claim this case study makes is untouched: an encoding that replaces an expensive composite comparison pays for itself, and in the very run that the baseline wins, RadixHuskySort is still 4.92x faster than the system sort on equal terms. What the result bounds is the recommendation. Two hundred thousand records give a parallel merge sort ample room to divide, while our own chunking does not reach eight threads until the whole corpus and builds a 65,536-entry histogram per chunk per pass at this digit width. So on a corpus of this size with cores to spare, *Arrays.parallelSort* is the right call, and closing that gap is a matter of our implementation rather than of the encoding.

A.11 Why the two Chinese corpora sit at opposite ends of the table. Both corpora are Chinese text, yet they sit at opposite ends of Table 6.2. The difference is entirely in how the two coders spend their 64 bits.

The Unicode coder gives each character sixteen bits, drawn from a range of thousands of code points, and a Leipzig Chinese word runs to one to four characters

— so the code often captures the whole word and the encoding is close to perfect. The pinyin coder gives each character nine bits for its syllable and two more for its tone, but there are only a few hundred distinct syllables and five tones, so the entropy available per character is far below sixteen bits however many are allocated to it. Names are two or three characters, and their surnames are drawn from a smaller pool again: a few hundred cover most of the corpus.

Distinct codes are therefore scarce where the Unicode coder has them in abundance, ties are frequent, and step 3 has genuine work to do rather than none. The pinyin case pairs the paper's most expensive comparison with its weakest encoding, which is why it is the smallest margin we report — and why the two Chinese rows should not be read as two points on a single axis.

A.12 Future work: composite keys, and what a library would have to provide. The permit coder above was written by hand, as was every composite encoding in this paper. One practical obstacle stands between that and a library implementation of this mechanism, and removing it is the piece of future work we think matters most for adoption. A hand-written coder must satisfy three properties that nothing checks. Fields must be packed most-significant-first in the same order as the type's own comparison. Symbol codes must run from one upwards, so that a field shorter than its width pads with zero and sorts low, as a string prefix does. And a symbol outside a field's alphabet must collapse to the largest symbol at or below it, so that an unexpected value weakens the ordering rather than inverting it.

The first of those is the one that bites, and it bites hardest where one might expect it to matter least. An encoding declared perfect skips the cleanup pass entirely (§ 3.4), so if its field order does not match the comparison, the sort returns the wrong answer and nothing downstream detects it. The same mistake in an imperfect encoding is merely slow: the cleanup pass re-sorts by the real comparison and the answer is still correct. Field order therefore matters most exactly when the encoding is at its best.

We expect most of this can be automated rather than merely assisted. Where a type's ordering is already derived from its field declarations — as with Java records, or Python dataclasses declared with an ordering — the encoding can be derived from the same declaration, which removes by construction the possibility that packing order and comparison order disagree, and would let the perfection flag be computed rather than asserted and separately verified. What such a derivation additionally needs is each field's range rather than its type, since it is the narrowness of the fields that lets a composite key fit at all: the permit block field is five characters of a thirty-one symbol alphabet, not an unbounded string. That range is either declared or inferred from the data.

This argues specifically for the radix variant. Once the fields do not fit in 64 bits — three fit comfortably, six will not — a wider code costs RadixHuskySort proportionally more digit passes and nothing else (§ 3.2), where a comparison-based husky sort has no comparable escape.

A.13 Adversarial inputs. The discussion in section A.1 raises a concern: it may be possible to construct inputs on which the husky encoding has poor entropy, and it is not obvious in advance how gracefully (or badly) the sort that follows would degrade in that case. We constructed two such inputs deliberately, to see how the two variants (QuickHuskySort and RadixHuskySort, § 3.2) each degrade.

Collapsed high-order bits. We generated arrays of *Long* values with a swept number of high-order bits held identical across every element (the rest random) — 0 fixed bits is the random baseline, 63 leaves only the sign bit free — and compared against a reimplementation from scratch of dual-pivot quicksort (the same baseline used for the numeric comparisons above, without any three-way or equal-key handling). That baseline is reported twice, and the pair is the most informative thing in this section. Java's own *DualPivotQuicksort* bounds its recursion at 64 levels and falls back to heapsort beyond it; our transcription of the algorithm originally did not. Table A.9 reports both, at $N = 1{,}000{,}000$.

RHSort's timings stay essentially flat across the entire sweep, exactly as the model of § 3.2 predicts: each digit pass does the same fixed amount of work regardless of how the keys are distributed, so a pass over collapsed digits (everything lands in one bucket) costs no more than any other pass.

The unguarded baseline degrades by more than an order of magnitude at 56 fixed bits and then fails outright, exhausting the stack at 60 and 63. This is a well-known failure mode for a quicksort that partitions naïvely around a heavily-duplicated pivot value: Bentley and McIlroy's classic treatment of engineering a practical quicksort [4] specifically addresses this case (a solution to Dijkstra's Dutch National Flag problem, partitioning into three regions — less than, equal to, and greater than the pivot — rather than two), precisely to avoid the pathological recursion depth this baseline exhibits. The degradation is far worse than the crash makes it look, because the crash truncates it. Removing the stack limit rather than the recursion, and timing the same input at $N = 1{,}000{,}000$ with the depth bound

Table A.9: Timings (ms) as high-order bits are progressively collapsed, $N = 1{,}000{,}000$. The dual-pivot baseline is shown both without a recursion-depth bound and with the one the JDK itself applies.

Fixed high bits	System sort	QuickHuskySort	Dual-pivot quicksort		RHSort/16
			unguarded	guarded	
0	477.6	420.0	335.7	335.9	74.8
32	493.8	420.8	340.9	331.2	69.6
48	485.1	357.9	354.6	333.3	66.8
56	174.3	197.8	**7097.7**	717.2	58.9
60	115.9	69.8	*stack overflow*	805.5	42.6
63	42.6	15.4	*stack overflow*	186.9	34.9

raised until it never fires, gives 363 seconds at 63 fixed bits against 0.4 seconds at none — a factor of nine hundred, on data of identical size.

The guarded column is the same algorithm with the bound the JDK ships, and it is a different story entirely: nothing crashes, and the 21x degradation at 56 fixed bits becomes 2.1x. What that column is measuring, however, is largely *not* quicksort. On these inputs the partitions go unbalanced immediately, the bound is reached within the first few dozen levels, and heapsort finishes the work. The clearest evidence is the direction of the sweep: unguarded, the worst case is 63 fixed bits, the most degenerate input, which is what the partitioning pathology predicts; guarded, 63 is the *fastest* cell in the row. The guard has not made dual-pivot quicksort robust here. It has detected that dual-pivot quicksort cannot cope and changed algorithm, which is what a production sort should do and is worth stating plainly, since a reader may otherwise take the guarded column for evidence that the pathology is mild.

QuickHuskySort neither crashes nor slows down anywhere in this sweep — if anything it gets faster as duplication increases, the same pattern System sort shows, consistent with Timsort's adaptive run-detection treating long runs of nearly equal keys as already sorted. Indeed both overtake RHSort at the extreme of the sweep, where 63 fixed bits leave almost nothing to sort and an adaptive algorithm has almost nothing to do, while radix still pays for its fixed number of passes. Radix sort's robustness is of a third kind, and the most useful of the three: it needs neither an adaptive algorithm's luck with the input nor a fallback to a different algorithm, because its cost does not depend on the distribution at all. It is the only sort here whose worst case can be read off its best.

Shared string prefixes. We took real corpus words and prepended a fixed-length prefix of repeated characters, long enough in the worst case to exceed the English coder's character-capture window (nine 7-bit ASCII characters per 64-bit code). Table A.10 reports timings at $N = 1{,}000{,}000$.

This is a genuinely different failure mode, and a less flattering one. Once the shared prefix reaches the coder's nine-character window, every element's husky code becomes bit-for-bit identical — the encoding provides zero disambiguation — and *every* husky-based approach, radix included, loses its advantage completely, finishing level with plain System sort or a little behind it rather than ahead. The mechanism has nothing to do with which algorithm sorts the codes in step 2: once the encoding carries no information at all, the entire ordering burden falls on the mandatory cleanup pass (step 3) doing real string comparisons — comparisons that are themselves more expensive than usual, since they must scan past the shared prefix before finding a difference — and whichever algorithm ran first in step 2 did genuinely useless work before that. Radix does not recover any advantage here, and is very slightly the slowest of the three husky-based options in this regime, since its fixed per-pass cost (a strength above) becomes pure overhead once there is no information left to sort.

Taken together, these two experiments answer the question posed above with two distinct results. When keys collide in their high-order bits but the encoding itself is still informative, RHSort is effectively immune, and a comparison sort is not: unguarded it degrades by orders of magnitude or fails outright, and guarded it survives only by abandoning the algorithm one asked for. But when the source data overwhelms the encoding's fixed capture window entirely, no choice of sort algorithm for the encoded longs helps at all; this is a limitation of the fixed-width encoding scheme itself, not of HuskySort's algorithmic strategy, and it is already implicit in the p_{crit} discussion of § 3.4: whenever the actual input defeats the encoding, the result is that no downstream sort algorithm can recover the information the encoding failed to capture.

Worth noting as a related, if incidental, finding: the three-way radix quicksort baseline used in § 3.2 for the classic string-sorting-literature comparison (Multi-

Table A.10: Timings (ms) as a shared string prefix grows, $N = 1{,}000{,}000$

Prefix length	System sort	QuickHuskySort	RHSort/16
0	1351.6	583.4	234.4
10	862.1	867.0	873.0
20	861.7	912.7	904.1
40	887.6	944.0	967.4

keyQuicksort) shares exactly the same failure mode as the naive quicksort baseline above, for a similar structural reason. Its equal-partition recursion advances one level per shared prefix character; on a long enough run of strings sharing a common prefix — the same kind of adversarial construction as above — a straightforward recursive implementation consumes one stack frame per shared character and eventually overflows the stack. Unlike the failure discussed above, this is a property of the comparison algorithm itself and unrelated to husky encoding. We fixed it in our own implementation by converting that one recursive call into a loop, restoring constant stack depth regardless of prefix length.

The MSD radix sort baseline needs a comparable fix, for a different reason. A run of equal strings occupies a single bucket at every depth beyond their length, so recursion into that bucket reduces neither the range nor the work outstanding. The cutoff to insertion sort conceals this below its own threshold, and a corpus sampled with replacement exceeds that threshold at scale. All three of the comparison baselines discussed here therefore carried an unbounded-recursion failure mode, and all three have now been repaired: the multikey recursion converted to a loop, the MSD cutoff corrected, and the dual-pivot baseline given the depth bound the JDK applies. Only the third of those changes what the sweep reports, which is why it is shown both ways. RadixHuskySort was never at risk of any of them in the first place, since LSD radix sort has no recursion to begin with.

A.14 Handling of partially-sorted arrays. Not surprisingly, QuickHuskySort is best suited for sorting random arrays. The benefits of partially-sorted arrays are enjoyed most by the merge sort family of algorithms (including Timsort). However, we have studied the efficacy of a modified merge sort (it is required to manage both the objects and the longs) as step 2. For random English String inputs, the merge-sort variation improves on the system sort by approximately 20 percent, where the normal Introsort-based QuickHuskySort improves on it by more like 40 percent.

However, neither husky variant is recommended when arrays are partially ordered. In such cases, the merge-sort variation of QuickHuskySort will take between one and a half and four times as long as Timsort.